

Software Testing and Quality Assurance

Lecture 4

Fabian Fagerholm

fabian.fagerholm@aalto.fi



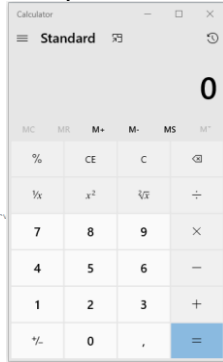
Aalto University
School of Science



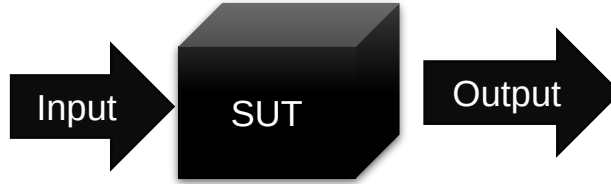
Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3

Concepts

System Under Test (SUT): The software system being tested



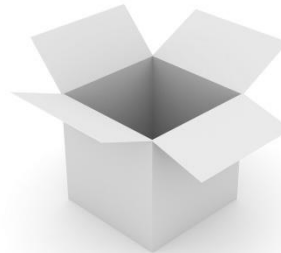
```
// Copyright (c) Microsoft Corporation. All rights reserved.  
// Licensed under the MIT license.  
  
using CalculatorApp.ViewModel.Common;  
using System;  
  
namespace CalculatorApp  
{  
    namespace Converters  
    {  
        /// <summary>  
        /// Value converter that translates true to false and vice versa.  
        /// </summary>  
        [Windows.Foundation.Metadata.WebHostHidden]  
        public sealed class RadixToStringConverter : Windows.UI.Xaml.Data.IValueConverter  
        {  
            public object Convert(object value, Type targetType, object parameter, string language)  
            {  
                // ...  
            }  
        }  
    }  
}
```



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

Grey-box testing: Black-box + White-box

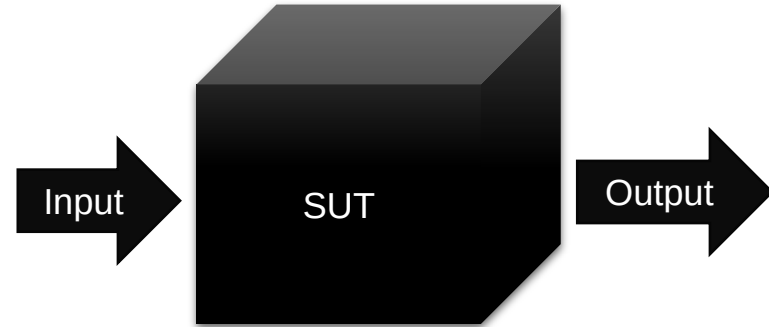


White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing*)

Black-box testing

Black-box testing

- Testing that does not examine the internals of the SUT
 - Inputs
 - Outputs
- Also called *functional testing*, *behavioural testing*, or *specification-based testing*
- Can be applied on all levels of testing (unit, integration, system, acceptance)
- Automated tests can be black-box tests
- Exploratory testing is a kind of black-box testing
- Advantage: can be efficient and effective
- Disadvantage: can never be sure of how much of the system has been exercised
- We focus here on general techniques that can be applied in more rigid tests (both manual and automated)



Some very specific inputs could lead to a rare execution path that cannot be known without seeing the code

```
if (student.number == 1234567) {  
    student.grade = 5;  
}
```

Techniques for designing black-box tests

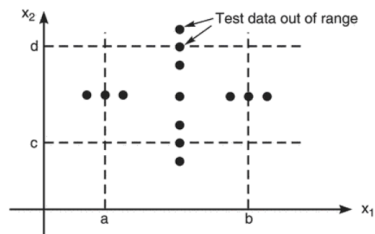
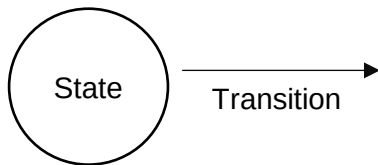


FIGURE 4.2 Robustness Test Cases.
Chopra (2018)

Boundary value analysis
Robustness testing

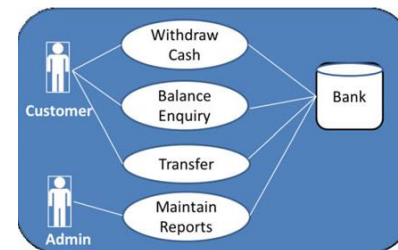


State transition testing

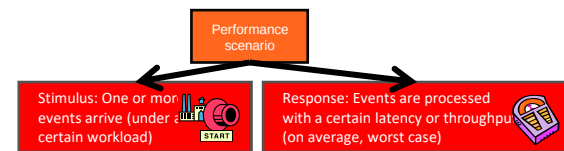
	Rule 1	Rule 2	Rule 3	Rule 4
<i>Input conditions</i>				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
<i>Actions</i>				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No

Test case 1 Test case 2 Test case 3 Test case 4

Decision table testing



Use case based testing



Latency: When the researcher presses Start, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

Scenario based testing

Equivalence partitioning

- Divide test input data into a set of classes
- Select one input value from each class
- The input value represents the class
- Create one test with each chosen input value
- → One test per class
- Significantly reduces number of test cases
- Keeps reasonable test coverage

Example

- Inputs: integers 1-100
- 100 tests?
- With equivalence partitioning:

Invalid low class:
A number below 1

Valid class:
A number between
1 and 100

Invalid high class:
A number above
100

Equivalence class testing: two approaches

- Testing-by-contract
 - Modules are designed to work under certain preconditions
 - Create test cases only for situations where preconditions are met
 - No test cases for conditions that the module does not claim to work under
- Defensive-testing
 - Modules are designed to accept any input
 - Create test cases for both valid and invalid preconditions

Contract for `openFile(filename)`

- Will return a file handle if `filename` exists
- Behaviour if `filename` does not exist: undefined
- Test case: use an existing filename

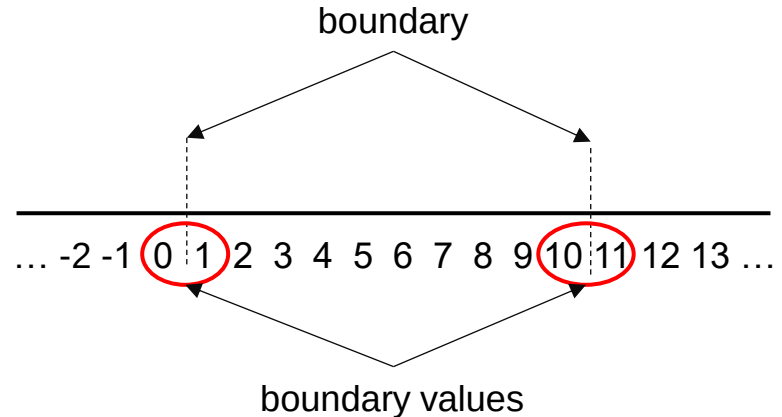
Contract for `openFile(filename)` using defensive design

- Will return a file handle if `filename` exists
- Will return an error if `filename` does not exist
- Test case 1: use an existing filename
- Test case 2: use a non-existing filename

Boundary value analysis (BVA)

- Assumption: defect density is clustered towards value range boundaries
 - Errors in implementing comparisons ($<$ vs. $<=$)
 - Errors in implementing loop termination conditions
 - Errors in interpreting requirements at the boundaries
- Test cases can concentrate on these boundary values
- Identify equivalence classes
- Identify boundaries of each equivalence class
- Create test cases for:
 - One point just above the boundary
 - One point just below the boundary

Example:
Valid inputs: integers 1-10



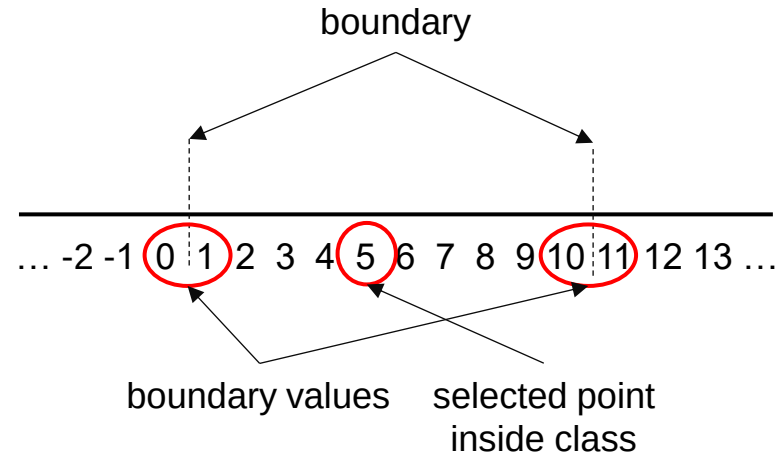
Robustness testing (variant of BVA)...

Some define
BVA as
robustness
testing

- Testing across the valid and invalid partitions
- Create test cases for:
 - One point just above the boundary
 - One point just below the boundary
 - One point inside the class
- BVA and its variants can be used to generate systematic tests and reduce the number of test cases
- Multiple parameters: only change one parameter at a time

Example:

Valid inputs: integers 1-10



Decision table testing

- Decision table:
 - A systematic way of recording complex business rules
 - Serves as a guide to creating test cases
- List all *input conditions* that can occur and all *actions* that can arise from them
 - Input conditions on the top
 - Resulting actions on the bottom
 - Each column represents a business *rule*
 - Each column becomes a test case
- If the conditions are value ranges, consider testing at both the low and high end of the range (compare: BVA/robustness testing)
- Every possible combination does not need to be entered, only those that represent actual business rules

	Rule 1	Rule 2	Rule 3	Rule 4
<i>Input conditions</i>				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
<i>Actions</i>				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No

Test case 1

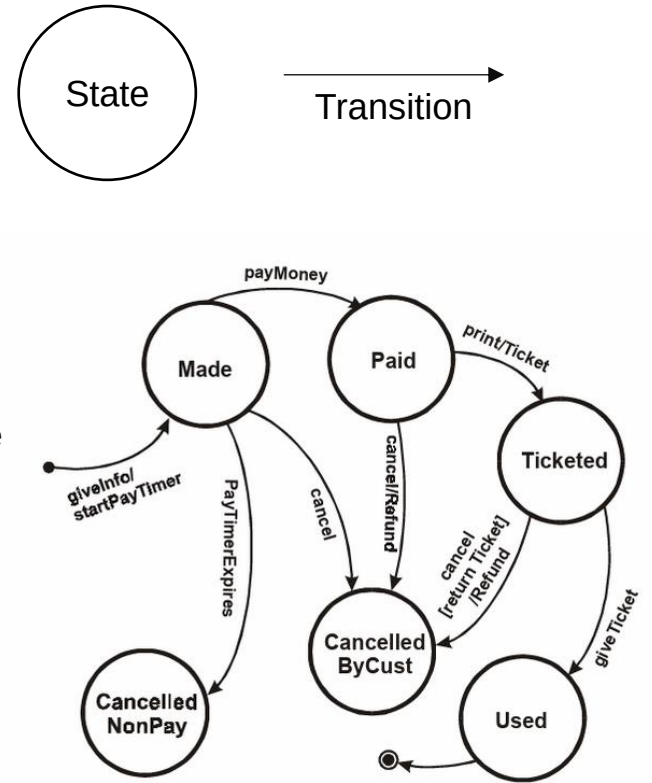
Test case 2

Test case 3

Test case 4

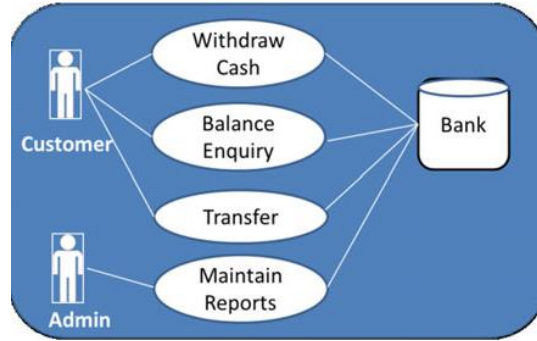
State transition testing

- Sometimes outputs depend on system state: system behaviour is triggered by state transitions
- State: a “stable” system condition – does not change unless a trigger event occurs
- Transition: a description of the trigger that moves the system from one state to another
- Create a state diagram of the system’s states and transitions
- Can also create a state table
- Test cases can be created depending on desired coverage
 - All states visited at least once (weak)
 - All events triggered at least once (weak)
 - All paths executed at least once (stronger)
 - All transitions exercised at least once (strongest)



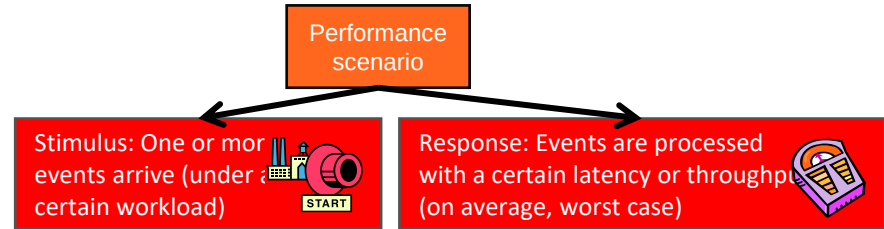
Copeland. (2003). A Practitioner's Guide to Software Test Design.

Use case and scenario-based testing



- Use cases often lack necessary detail for testing
- Add details, e.g.
 - Preconditions
 - Main success scenario
 - Success end conditions
 - Failed end conditions
 - Response time
 - Frequency
 - ...

A detailed specification is an important input to test design



Latency: When the researcher presses *Start*, the simulation world with 100 creatures is initialized and animation begins within an average response time of one second.

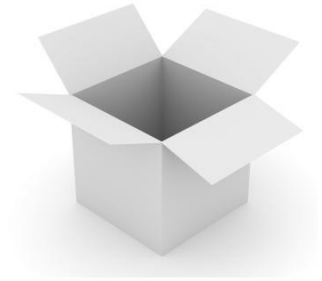
Throughput: With 100 interacting creatures, the system can process, store and visualize at least 100 simulation steps per minute.

The beginnings of a performance test case

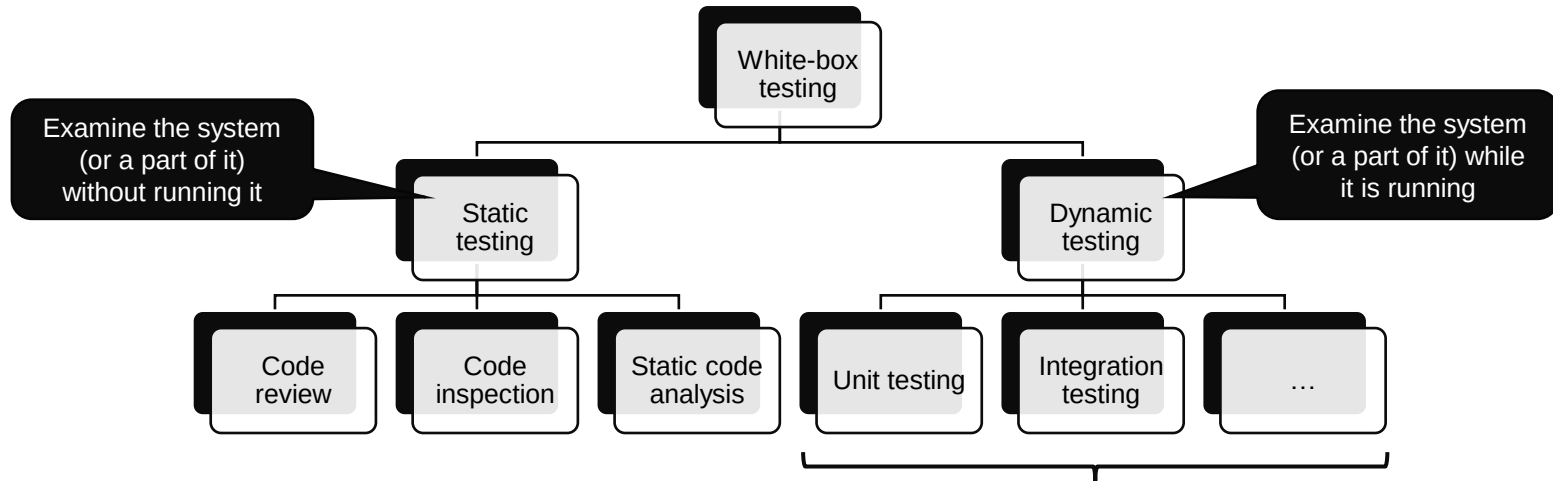
White-box testing

White-box testing

- Also: clear-box, glass-box, or *structural testing*
- Designing tests for the SUT *using knowledge of its internal structure*
- *Today: techniques for designing test cases based on a white-box approach*



[Photo CC BY-SA-NC](#)



Potentially all levels of the testing pyramid – just like in black-box testing but based on knowledge of the internal structure

Dynamic white-box testing

- Creating test cases using knowledge of internal structure:
 - Branches
 - Individual conditions
 - Statements
- General approach
 - Analyse SUT implementation
 - Identify **paths** through SUT
 - Choose input to cause execution of selected paths
 - Determine expected results for selected paths
 - Run tests
 - Compare actual outputs with expected outputs
 - Make conclusions about correctness of SUT implementation

Emphasis is on code **paths**

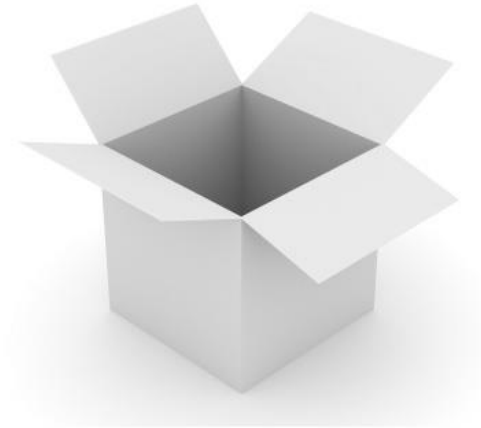
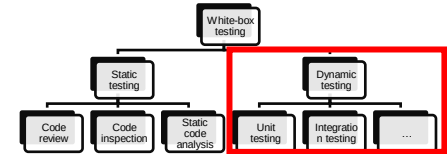


Photo CC BY-SA-NC



Dynamic white-box testing

- Possible applications on different testing levels
 - Unit: Test paths within individual modules before integration
 - Integration: Test paths between modules (also: data flow)
 - System: End-to-end application flow from user perspective
- Challenges
 - Cannot test all possible paths
 - Test cases may not detect data sensitivity errors (e.g., $p=q/r$ may execute correctly except when $r=0$)
 - Assumes control flow is correct (non-existent paths cannot be discovered)
 - The tester must have programming skills to evaluate the SUT

Dynamic white-box testing emphasises paths – test case design is based on path analysis

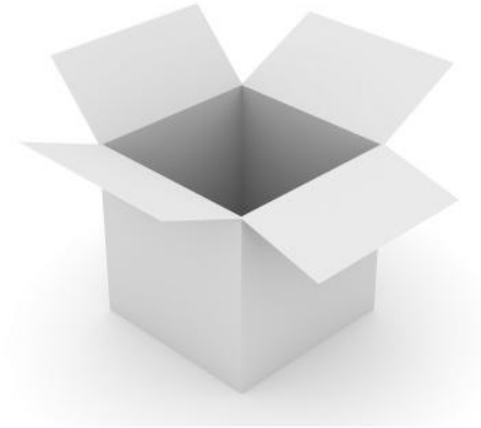
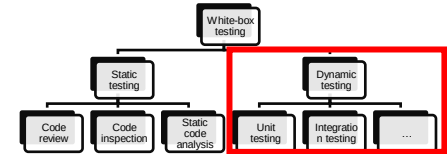


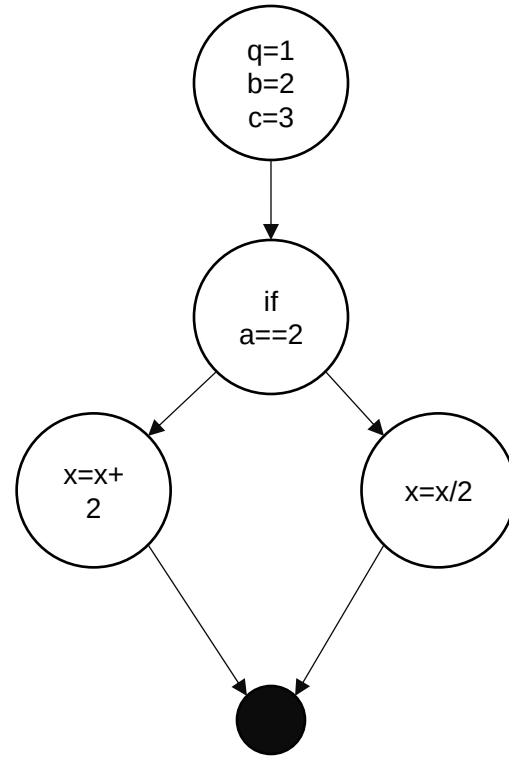
Photo CC BY-SA-NC



Control flow testing

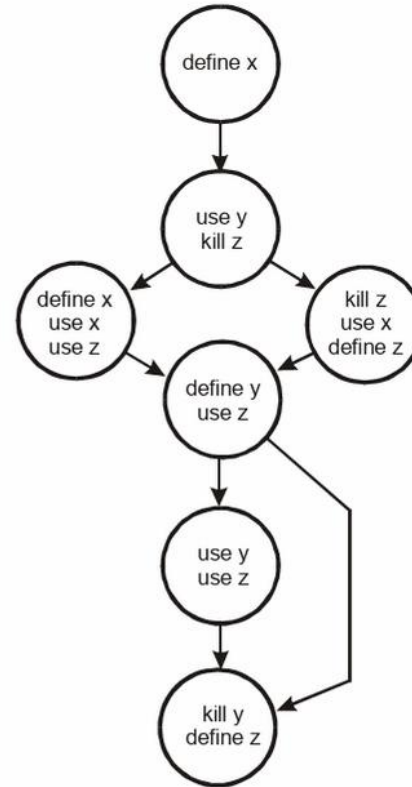
- Convert the code into a graph
- Analyse the paths through the graph
- Create test cases based on the analysis
- There are different possible levels of path coverage

```
q=1
b=2
c=3
if (a==2):
    x=x+2
else:
    x=x/2
```



Data flow testing

- Follow the creation, use, and destruction of variables, e.g.
 - Ensure that variables are not used before they are defined
 - Ensure that variables are destroyed
 - Ensure that variables are not used after they are destroyed
- Data flow testing tools can help
- Many compilers do data flow testing
- (Not covered in depth in this course)



(Copeland 2003)

Statement coverage

- What portion of statements are executed by our tests?
- Expression:
 - A syntactic unit in a programming language
 - May be *evaluated* to determine its value
 - Examples: `3 + 5` (evaluates to 8), `3 > 5` (evaluates to false)
- Statement:
 - A syntactic unit in a programming language (at least in imperative languages)
 - Performs some *action*
 - Example (simple): `y = 8`
 - Example: (compound):
`if (<condition>):`
`<sequence>`

(Statements in <sequence> counted separately)



Aalto University
School of Science

Statement coverage =

$$\frac{\text{Total statements executed}}{\text{Total statements in program}}$$

Q: How many statements does this code snippet have?

```
if (x > 5):  
    y = 8
```

A: Two statements.

Q: How many tests do we need to get 100% statement coverage?

A: One (any value for x that is larger than 5). To get 100% statement coverage, the condition has to be true.

Statement coverage

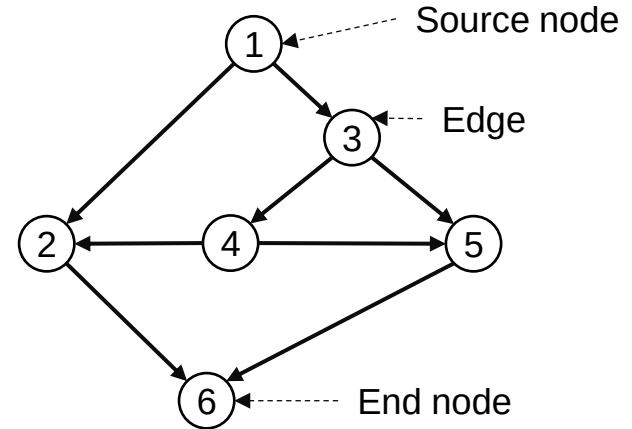
- In principle, we want 100% statement coverage, but ...
- For a set of sequential statements (usually easy, but):
 - Exceptions (e.g. divide by zero) will exit the code early
 - The code could be entered from multiple points
- For code with conditional branches (if-then-else two-way-choices):
 - One test case per branch is needed
- For code with loops, failures are often in boundary conditions. We need test cases that:
 - Skip the loop (termination condition is true before the loop)
 - Check the loop for one iteration ($n = 1$)
 - Check the loop for values close to n (boundary value analysis!)

Statement coverage =

$$\frac{\text{Total statements executed}}{\text{Total statements in program}}$$

Path coverage

- Create a graph representation of the program
 - Start from the beginning
 - Take any path to completion
- Stronger coverage condition than statement coverage
 - 100% statement coverage does not necessarily exercise all code paths
- The more paths are covered, the better
- But what about the nodes?



Path coverage =

$$\frac{\text{Total paths executed}}{\text{Total paths in program}}$$

Some possible paths:

P1: 1 → 2 → 6

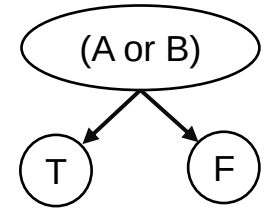
P2: 1 → 3 → 5 → 6

P3: 1 → 3 → 4 → 5 → 6

P4: 1 → 3 → 4 → 2 → 6

Decision coverage

if (A or B):
...



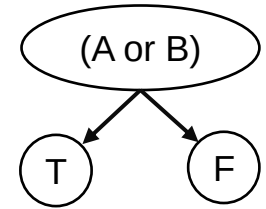
- Requirements for test:
 - There must be one test for each decision outcome (= edge in the graph)

A	B	(A or B)
T	F	T
F	F	F

- Problems
 - The effect of B is not tested: cannot distinguish between decision (A or B) and decision A
- Minimum of two tests per decision

Condition coverage

if (A or B):
...



- Requirements for test:
 - Each condition takes on all possible outcomes at least once
 - (Does not require decision to take on each outcome)

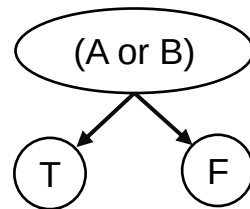
A	B	(A or B)
T	F	T
F	T	T

- Problems
 - The decision does not take on all possible outcomes
- Minimum of two tests per decision

Condition/decision coverage

if (A or B):

...



- Combination of condition and decision coverage
- Requirements for test:
 - There must be one test for each decision outcome
 - Each condition takes on all possible outcomes at least once

A	B	(A or B)
T	T	T
F	F	F

- Problems
 - Cannot distinguish the correct expression (A or B) from A or B or (A and B)

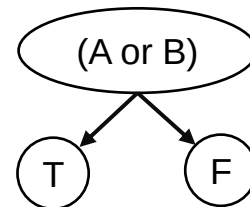
Modified condition/decision coverage

- Stronger coverage condition than condition/decision coverage
- Requirements for test:
 - Each entry and exit point is covered
 - Each decision takes every possible outcome
 - Each condition takes every possible outcome
 - Each condition must be shown to independently affect the outcome
- Minimum of $n+1$ test cases for a decision with n inputs

Trivial in this example

if (A or B):

...



A	B	(A or B)
T	F	T
F	T	T
F	F	F

Example for more complicated code

- Designing tests with the desired coverage can be hard!

```
def do_it(c1, c2, c3):  
    x = 0  
    if (c1):  
        x += 1  
    if (c2):  
        x -= 1  
    if (c3):  
        x += 3  
    return x
```

How many test cases are required?

What should the inputs be for each case?

Statement coverage

Check that all statements in the code are executed
For 100% coverage: 1 test case (true, true, true)

Condition coverage

Check that all possible results of conditions occur
For 100% coverage: 2 test cases (true, true, true), (false, false, false) – or any other combination where each conditional is tested once with true and once with false

Path coverage

Every possible path through the code is executed
For 100% coverage: 8 test cases – two alternatives for each conditional $\rightarrow 2 * 2 * 2 = 8$

Acceptance testing

- “[Testing], which when completed successfully, will result in the customer accepting the software and [paying for it]” (Copeland 2003)
- A set of test cases that are agreed between supplier and buyer to balance completeness and cost
- Conducted to determine if the requirements of a specification or contract are met
- User acceptance testing (UAT): verifying that the solution works for the user
- Operational acceptance testing (OAT): determine operational readiness (pre-release) of the software
- In Extreme programming and agile terminology:
 - Scenarios to test when a user story has been correctly implemented
 - Specified by the customer
- Acceptance tests are usually black-box tests

Acceptance testing with Robot Framework

- Test suite: one .robot file
- Contains one or more test cases
- That utilise keywords
- That can be defined in .resource files
- Or in built-in or custom libraries

```
*** Settings ***
Documentation    A test suite for valid login.
...
...             Keywords are imported from the resource file
Resource        keywords.resource
Default Tags    positive

*** Test Cases ***

Login User with Password
    Connect to Server
    Login User          ironman    1234567890
    Verify Valid Login   Tony Stark
    [Teardown]          Close Server Connection

Denied Login with Wrong Password
    [Tags]              negative
    Connect to Server
    Run Keyword And Expect Error    *Invalid Password    Login User    ironman    123
    Verify Unauthorised Access
    [Teardown]          Close Server Connection
```

Next steps

- Individual assignments in MyCourses
 - Quiz on robustness testing
 - Theory assignment for statement coverage and MC/DC
 - Unit testing 2
 - Acceptance testing 1

Use the related reading for more details.
Reserve enough time to set up your environment.

Deadline: before next lecture (Tuesday by 10:00)

- Getting help
 - Ask in the course chat, see Communication section in MyCourses
 - Personal questions by email



Study smarter, not harder!

- Plan: when and where
- Go deep at your own pace
- Get some rest
- Practice makes perfect
- Enjoy it ☺